گزارش پروژه یادگیری ماشین
امیرمهدی واله - ۴۰۴۱۳۳۴۰۴۴

۱) مجموعه‌داده و پیش‌پردازش (Dataset & Preprocessing)

مجموعه‌داده

در این پروژه از مجموعه‌داده‌ی LOL-v1 استفاده شده است که شامل جفت‌تصاویر کم‌نور (Low-Light) و نور نرمال (Normal-Light) می‌باشد. تصاویر این مجموعه‌داده به‌صورت جفت‌شده ارائه شده‌اند و هر تصویر کم‌نور دارای نمونه‌ی مرجع با نور مناسب است. رزولوشن اصلی تصاویر تقریباً 600×400 پیکسل می‌باشد.

یش‌پردازش

به‌منظور کاهش هزینه‌ی محاسباتی و امکان آموزش مدل روی سخت‌افزارهای فاقد CUDA (مانند Apple Silicon)، تمامی تصاویر در مرحله‌ی پیش‌پردازش به یک رزولوشن ثابت تغییر اندازه داده شدند. تصاویر به بازه‌ی [0,1] نرمال‌سازی شده و به‌صورت یادگیری نظارت‌شده (Supervised Learning) مورد استفاده قرار گرفتند.

۲) معماری مدل (GLARE-Lite)

معماری مدل

این پروژه یک مدل سبک به‌نام GLARE-Lite را معرفی می‌کند که با الهام از ایده‌ی کلی روش GLARE طراحی شده است، اما برای آموزش محلی و منابع محدود بهینه‌سازی شده است. این مدل از ساختار Encoder–Decoder به سبک U-Net استفاده می‌کند و از اتصال‌های میان‌بُر (Skip Connections) برای حفظ جزئیات مکانی بهره می‌برد.

مدل به‌جای پیش‌بینی مستقیم تصویر نهایی، یک Residual Enhancement را تخمین می‌زند که به تصویر ورودی افزوده می‌شود. این استراتژی باعث پایداری آموزش و بهبود کیفیت بازسازی می‌شود. همچنین از Channel Attention (SE Block) برای بهبود نمایش ویژگی‌ها با سربار محاسباتی اندک استفاده شده است.

۳) تابع هزینه و هدف آموزش (Training Objective)

تابع هزینه پایه

در ابتدا، از تابع هزینه‌ی L1 برای آموزش مدل استفاده شد. اگرچه این تابع باعث بهبود روشنایی کلی تصویر می‌شود، اما مشاهده شد که خروجی‌ها در بسیاری از موارد دچار کاهش غلظت رنگ (Desaturation) می‌شوند، به‌خصوص در تصاویری با رنگ غالب (مانند قرمز).

تحلیل مشکل

تابع L1 تمایل دارد خروجی را به سمت مقادیر میانگین سوق دهد و در نتیجه، مدل برای کاهش خطا، شدت رنگ‌ها را کاهش می‌دهد. این رفتار منجر به تصاویر روشن ولی بی‌روح و کم‌رنگ می‌شود.

بهبود تابع هزینه

برای حل این مشکل، تابع هزینه با قیود رنگی گسترش داده شد:

- Exposure Loss: برای هم‌ترازی روشنایی کلی تصویر
- Chroma Loss: برای حفظ مؤلفه‌های رنگی (U و V) در فضای YUV
- Saturation Loss: برای حفظ دامنه‌ی اشباع رنگ‌ها

فرم نهایی تابع هزینه به صورت زیر است:

$$\mathcal{L} = \mathcal{L}_{L1} + \lambda_{exp}\mathcal{L}_{exp} + \lambda_{ch}\mathcal{L}_{chroma} + \lambda_{sat}\mathcal{L}_{sat}$$

این طراحی مانع از آن می‌شود که مدل با کاهش شدت رنگ‌ها، به‌صورت میان‌بُر (Shortcut) مقدار خطا را کاهش دهد.

۴) مشاهدات و مطالعه‌ی حذف (Observations & Ablation Study)

مشاهدات اولیه

آموزش مدل تنها با استفاده از L1 و Exposure Loss منجر به بهبود روشنایی تصویر شد، اما تصاویر خروجی از نظر اشباع رنگ به‌طور محسوسی ضعیف بودند. افزایش رزولوشن ورودی باعث بهبود کیفیت مکانی شد، اما مشکل کاهش غلظت رنگ را به‌طور کامل برطرف نکرد.

مطالعه‌ی حذف (Ablation Study)

با افزایش وزن Chroma Loss و افزودن Saturation Loss، تصاویر خروجی به‌طور قابل‌توجهی از نظر شادابی رنگ و حفظ شدت رنگ‌های غالب بهبود یافتند، بدون آن‌که روشنایی کلی تصویر آسیب ببیند.

۵) نتایج (Results)

نتایج کیفی

مقایسه‌ی بصری تصاویر نشان می‌دهد که مدل GLARE-Lite قادر است تصاویر کم‌نور را به‌طور مؤثر روشن کند و هم‌زمان شدت رنگ‌ها را بهتر حفظ نماید. به‌ویژه در نواحی با رنگ غالب (مانند لباس‌های قرمز)، نسخه‌ی بهبودیافته‌ی تابع هزینه منجر به خروجی‌هایی زنده‌تر و طبیعی‌تر شده است.

نمونه‌ای از نتایج به‌صورت مقایسه‌ی تصویر کم‌نور، خروجی مدل و تصویر مرجع در گزارش گام به گام ارائه شده است.

۶) محدودیت‌ها (Limitations)

با توجه به محدودیت منابع محاسباتی، مدل در یک رزولوشن ثابت آموزش داده شده است و تفکیک صریح مؤلفه‌های روشنایی و بازتاب (Illumination / Reflectance)

انجام نشده است. همچنین، به دلیل سبک بودن معماری، مدل ممکن است در شرایط نوری بسیار شدید به اندازه‌ی مدل‌های سنگین‌تر عملکرد نداشته باشد.

۷) جمع‌بندی (Conclusion)

در این پروژه، یک مدل سبک برای بهبود تصاویر کم‌نور با الهام از GLARE ارائه شد. نتایج نشان می‌دهد که طراحی مناسب تابع هزینه، به‌ویژه افزودن قیود رنگی، نقش مهمی در جلوگیری از کاهش اشباع رنگ‌ها دارد. مدل پیشنهادی با توجه به سادگی و کارایی، گزینه‌ای مناسب برای پیاده‌سازی روی سخت‌افزارهای محدود محسوب می‌شود.

۸) گزارش گام به گام:

از آنجایی که دسترسی به سرورهای خارجی محدود شد و این پروژه نیاز به پردازنده های قوی داشت، امکان راه اندازی پروژه Glare با سورس کد های خود آن مقدور نبود چرا که به GPU های قوی nvidia و همچنین محیط CUDA نیاز داشت. بر آن شدیم که به نحو دیگری پروژه را پیاده سازی کنیم.

ما تصمیم گرفتیم یک نسخه سبک از GLARE بسازیم که بتوانیم روی mps یک دستگاه مک بوک m5 با gpu و cpu محدود اجرا کنیم.

برای اینکار از همان تئوری glare استفاده کردیم ولی اجزا سنگین و پر هزینه آن (برای مثال Convolution) را حذف یا با بلاک های دیگر جایگزین کردیم.

توابع loss مورد استفاده:

L1 loss → pixel differences

Exposure loss → brightness mismatch

گام ۱:

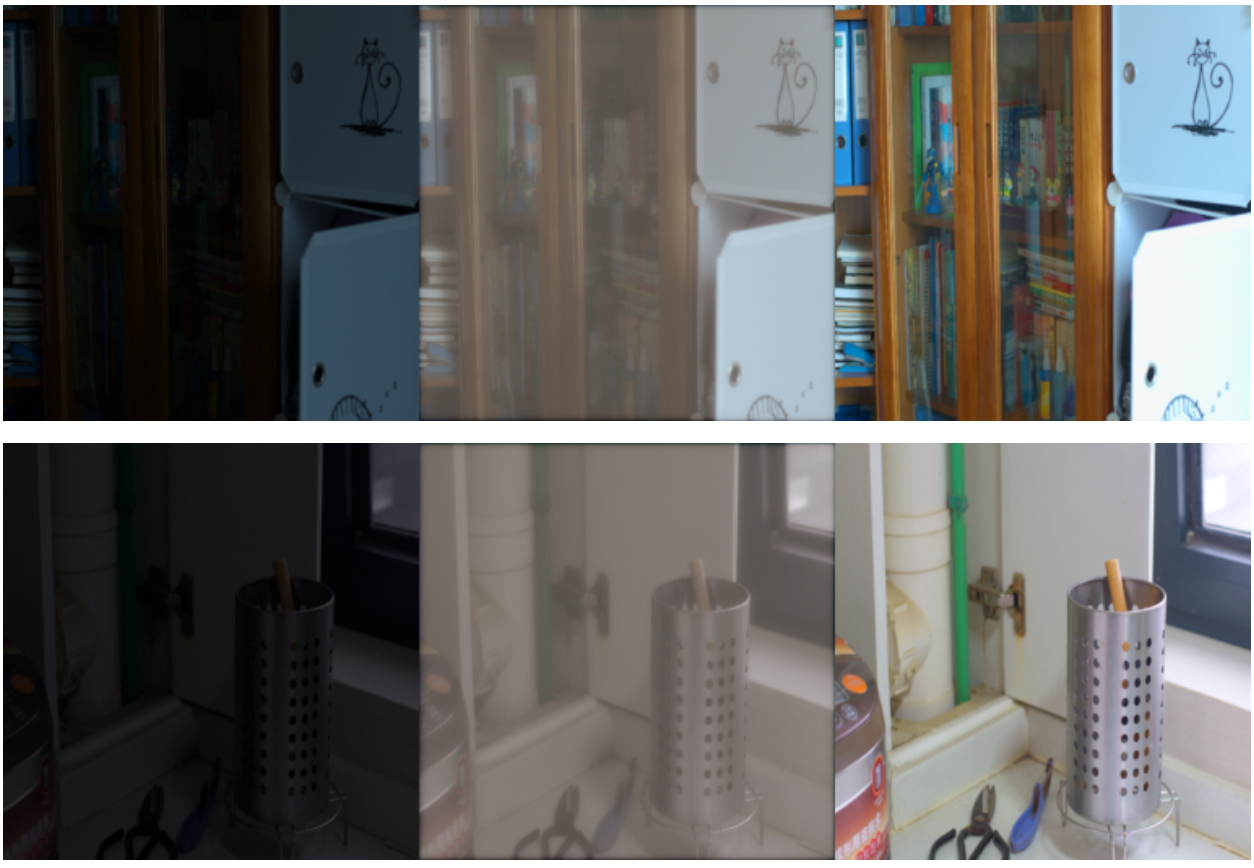
بار اول روی دیتاست LOL-v1 مدل را آموزش دادیم.
دستور:

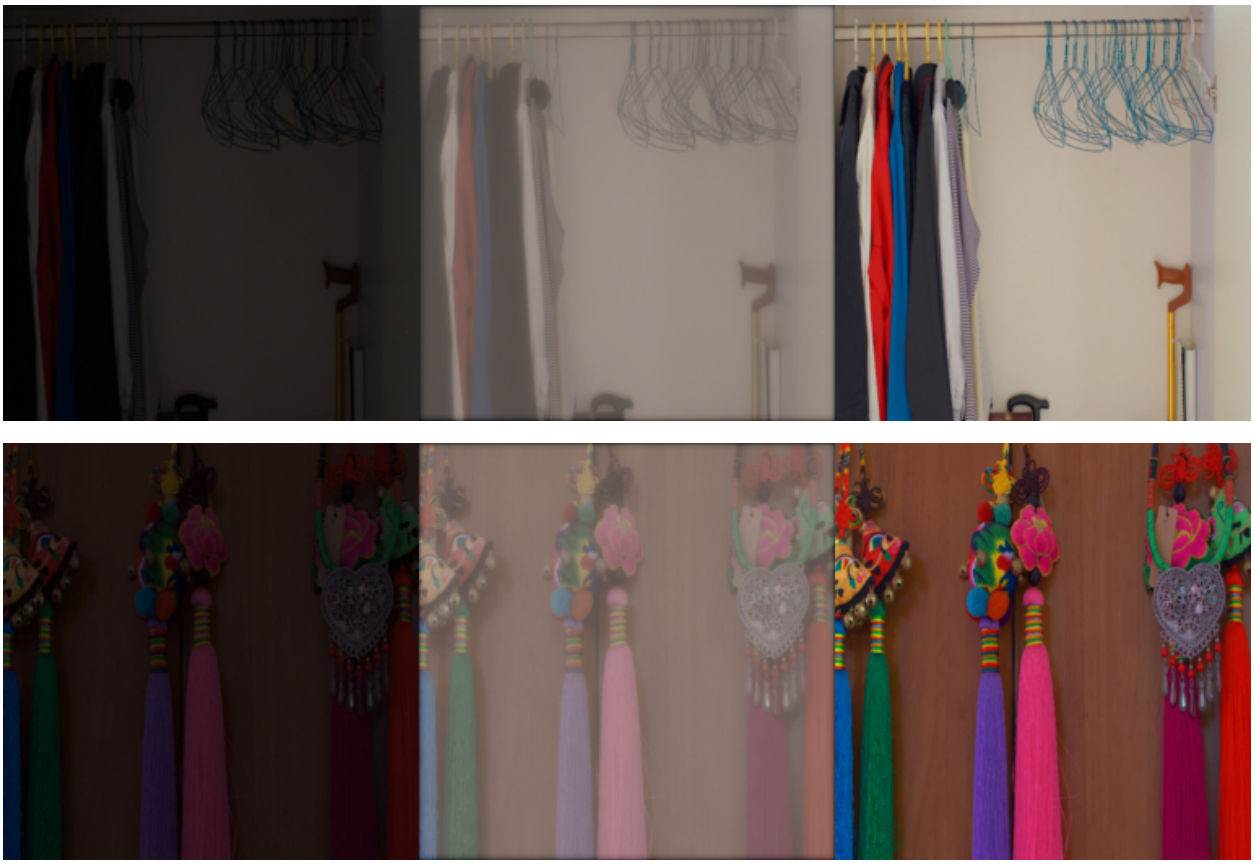
```
python glare_lite.py \   --train_dir data/train \   --val_dir data/val \   --epochs 1 \   --batch 2 \   --size 256 \   --out runs/glare_lite
```

خروجی که گرفتیم:

```
Epoch 1/1 - loss: 0.21583
```

نمونه خروجی (تصویر ترکیب شده: عکس اصلی | عکس بهبود یافته | عکس ورودی):





چون تعداد داده آموزش کم بوده است، تصاویر میانی، کمی مات و کدر هستند. در گام های بعدی تعداد آزمایش را بیشتر می کنیم.

گام ۲:

اینبار مدل را ۲۰ بار آموزش دادیم.

تابع loss نهایی بعد از هر بار:

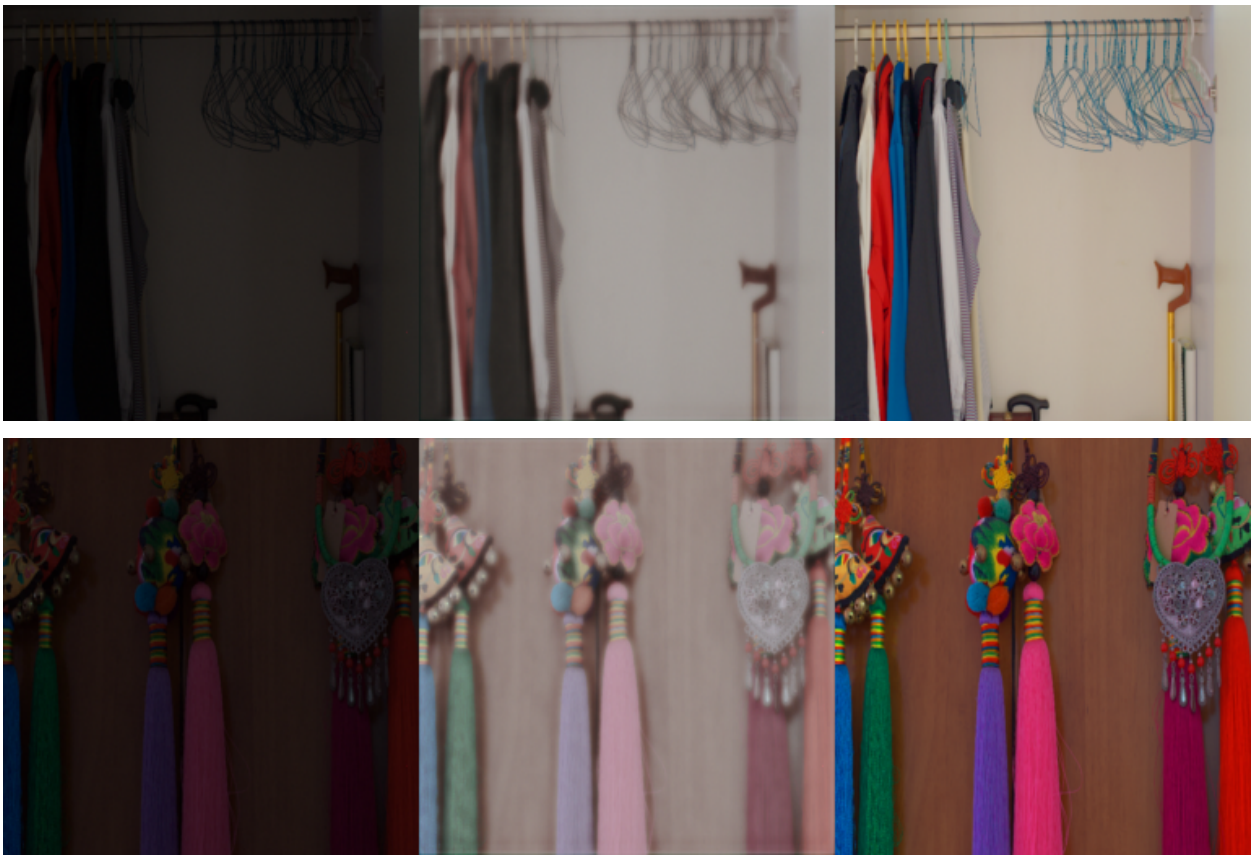
Epoch 1/20- loss: 0.25080
Epoch 2/20 - loss: 0.18850
Epoch 3/20 - loss: 0.17463
Epoch 4/20 - loss: 0.13203
Epoch 5/20 - loss: 0.12724
Epoch 6/20 - loss: 0.12407
Epoch 7/20 - loss: 0.12112
Epoch 8/20 - loss: 0.11878
Epoch 9/20 - loss: 0.11793

```
Epoch 10/20 - loss: 0.11640
Epoch 11/20 - loss: 0.11450
Epoch 12/20 - loss: 0.11411
Epoch 13/20 - loss: 0.11333
Epoch 14/20 - loss: 0.11367
Epoch 15/20 - loss: 0.11326
Epoch 16/20 - loss: 0.11390
Epoch 17/20 - loss: 0.11321
Epoch 18/20 - loss: 0.11165
Epoch 19/20 - loss: 0.11272
Epoch 20/20 - loss: 0.11279
```

همانطور که می بینیم، تابع loss بعد از هر بار اجرای cnn کمتر می شود.

خروجی های این گام:





کیفیت خروجی به عکس واقعی نزدیک تر شد.
اما الان کمی حالت پیکسل پیکسل می بینیم. به این مشکل Pixelation می گویند که بخاطر رزولوشن کم و بخاطر 256x256 انتخابی خودمان است.

در گام بعدی، با رزلوشن بالاتری آزمایش می کنیم.

تابع loss کنونی:

loss = l1 + 0.1 * lexp

گام ۳:
تست با ۳۲۰ پیکسل
دستور ترمینال:

```
python glare_lite.py --train_dir data/train --val_dir data/val --epochs 20 --batch 4 --size 320 --out runs/glare_lite_e20_s320
```

مقادیر loss:

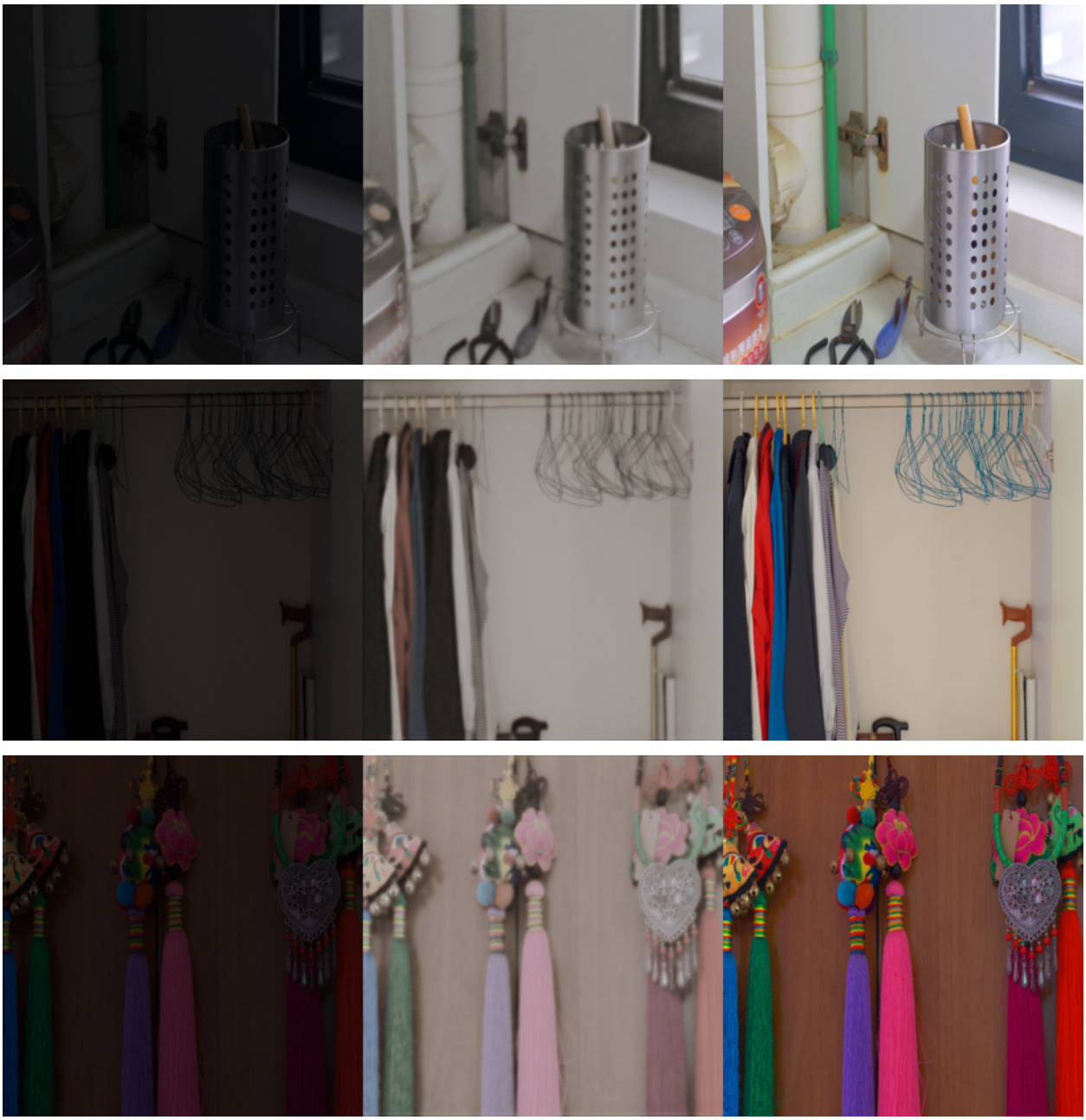
Epoch 1/20 - loss: 0.25495

Epoch 2/20 - loss: 0.17662
Epoch 3/20 - loss: 0.14620
Epoch 4/20 - loss: 0.12827
Epoch 5/20 - loss: 0.12318
Epoch 6/20 - loss: 0.12109
Epoch 7/20 - loss: 0.11936
Epoch 8/20 - loss: 0.11765
Epoch 9/20 - loss: 0.11572
Epoch 10/20 - loss: 0.11865
Epoch 11/20 - loss: 0.11512
Epoch 12/20 - loss: 0.11359
Epoch 13/20 - loss: 0.11293
Epoch 14/20 - loss: 0.11335
Epoch 15/20 - loss: 0.11284
Epoch 16/20 - loss: 0.11321
Epoch 17/20 - loss: 0.11375
Epoch 18/20 - loss: 0.11159
Epoch 19/20 - loss: 0.11164
Epoch 20/20 - loss: 0.11097

می بینیم که تابع loss به مقدار ۰.۱۱۰۹۷ رسید. در گام قبلی تابع loss برابر با ۰.۱۱۲۷۹ بود، یعنی ۰.۰۰۲ کمتر.

تصاویر خروجی این گام:





همانطور که انتظار می رفت، رزولوشن بالاتری در خروجی ها میبینیم. در تصویر چوب لباسی ها، این به وضوح قابل مشاهده است.

حال آنچه میتوانیم ببینیم اینست که خروجی ما غلظت رنگ کمتری نسبت به تصاویر اصلی دارد.
در واقع مدل color را خیلی با احتیاط اعمال کرده است.
یعنی مدل رنگ قرمز - صورتی - قهوه ای - سفید - آبی را **کمتر** قرمز - صورتی - قهوه ای - سفیر - آبی کرده است.

برای این کار یک تابع loss که رنگ را نیز در نظر میگیرد اضافه میکنیم (color loss) و به آن ۵ صدم وزن می دهیم که در تابع اصلی loss افزوده شود.

تابع loss کنونی:

loss = l1 + 0.1*lexp + 0.05*lch

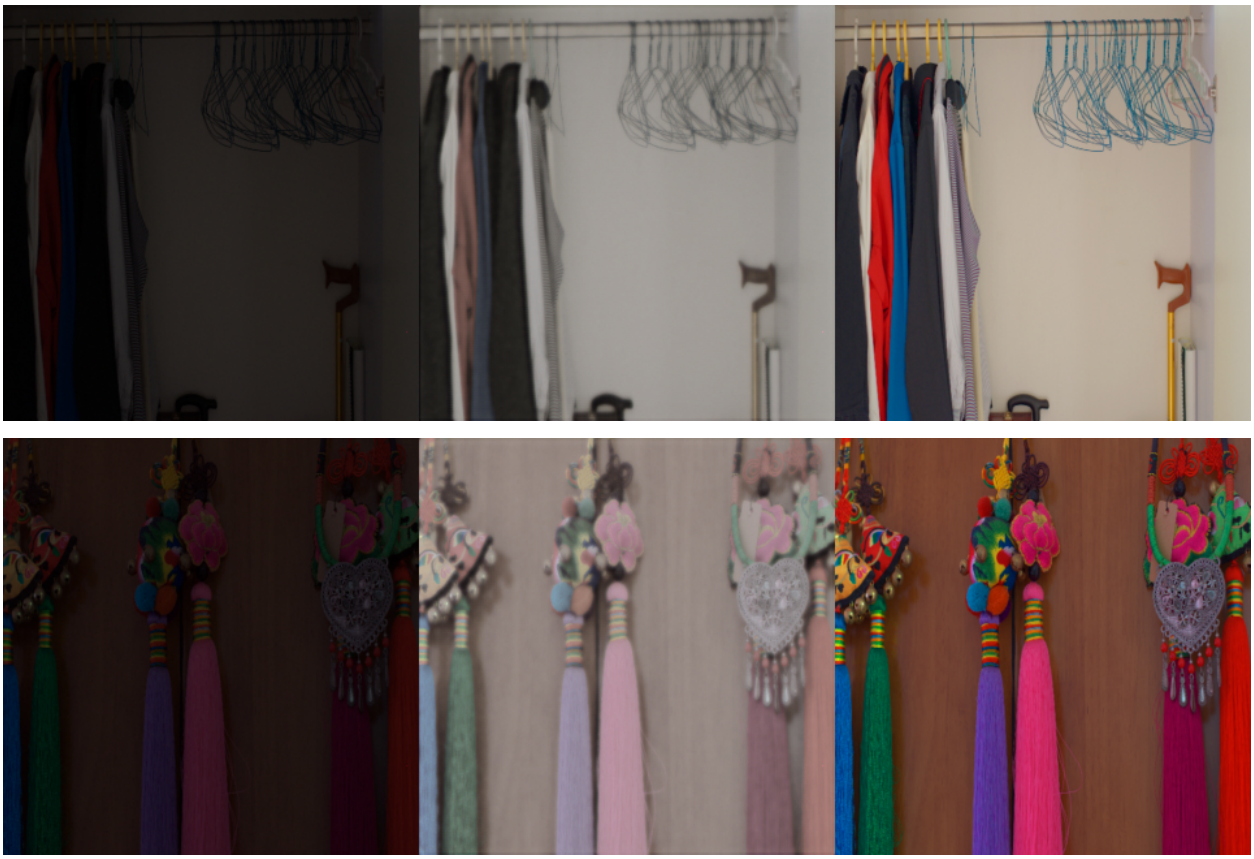
حال مجدد در گام بعدی با همان رزلوشن ۳۲۰ و ۲۰ بار تکرار و ۴ batch اجرا میکنیم.

گام۴:

با حضور Color Loss:

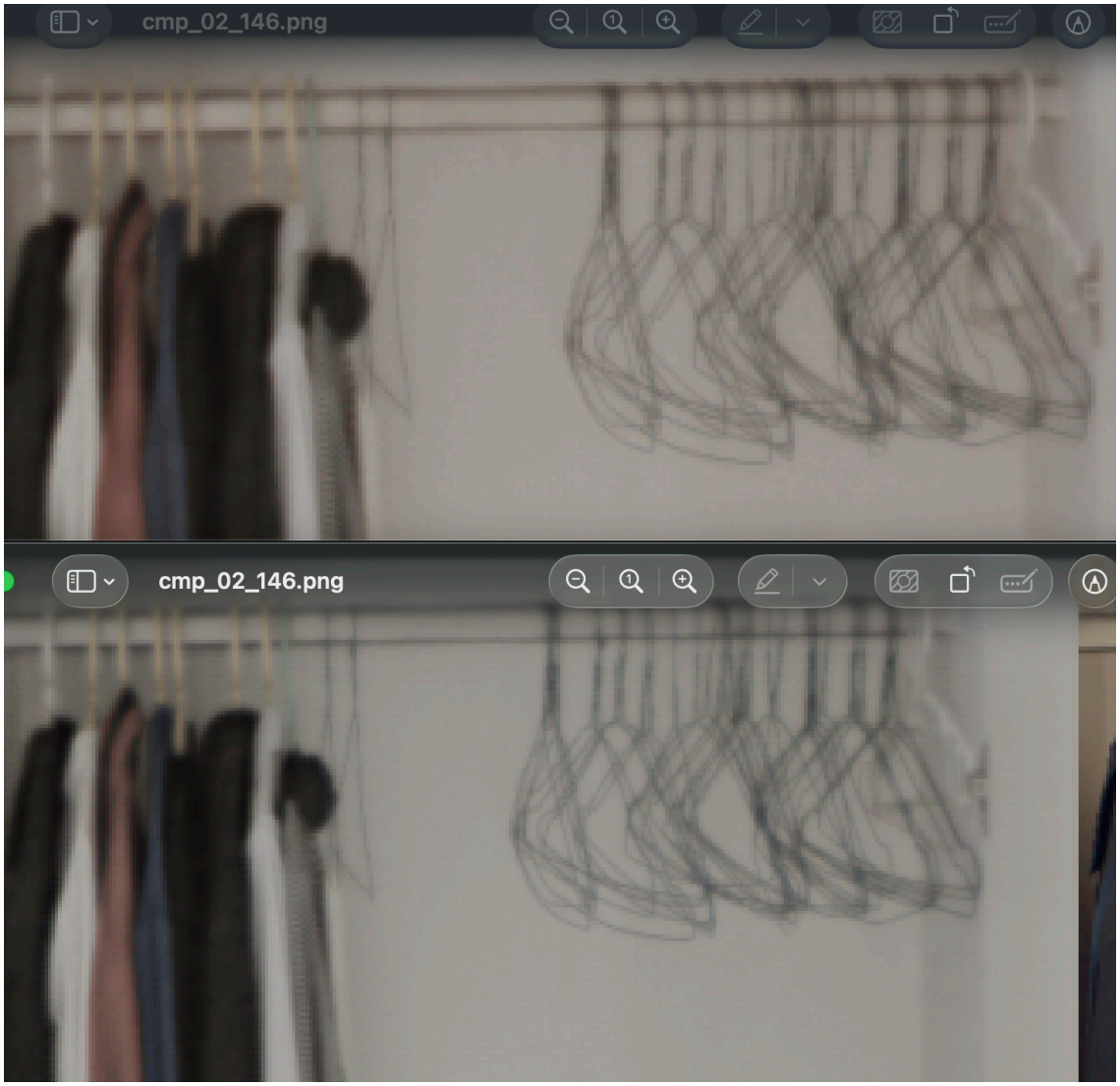
خروجی ها:





میزان خروجی نور ها مناسب است. بنظر می آید که به **اندازه کافی** هنوز مدل اشباع رنگ ها را در نظر نمی گیرد.

برای مقایسه راحت تر ۲ تصویر را (بالایی بدون loss رنگ و پایینی با loss رنگ) از گام قبلی با هم مقایسه می کنیم:



رنگ ها به مقدار خیلی اندکی بهبود یافته اند. در گام بعدی، از تابع loss حساسیت نور را کم می کنیم و به حساسیت رنگ می افزاییم.

همچنین یک تابع دیگر اضافه می کنیم که مستقیما Saturation را نیز اندازه گیری کند.

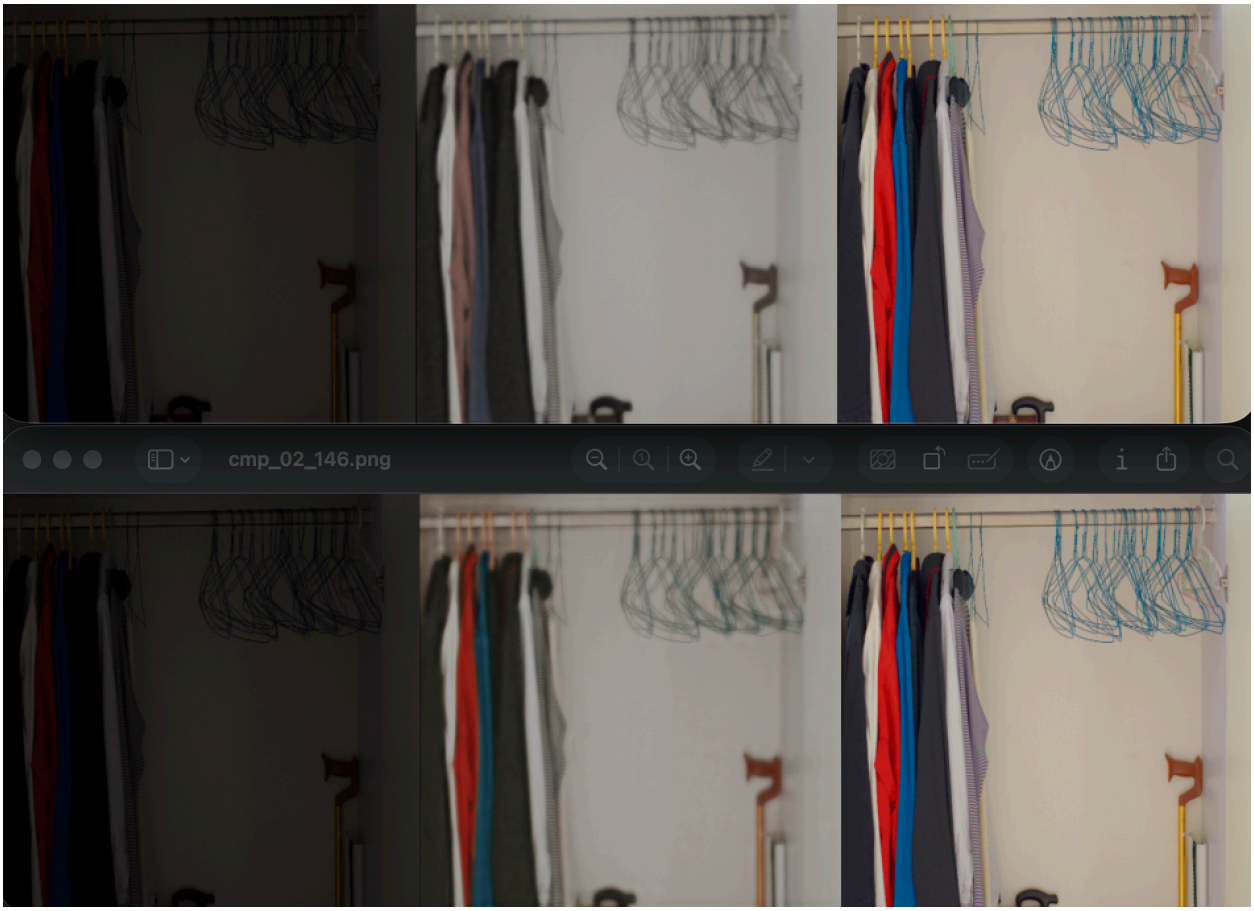
تابع loss بعدی:

$$loss = l1 + 0.05*lexp + 0.30*lch + 0.10*lsat$$

در گام بعدی، بجای ۲۰ epoche روی ۳۰ epoche اجرا می کنیم.

گام ۵: با وجود Saturation Loss:

تصویر بالا: خروجی گام ۴ (بدون Saturation loss و با ضریب کمتر در Color و بیشتر در نور)
تصویر پایین: خروجی گام ۵ (با Saturation loss و با ضریب بالاتر Color و نور کمتر)



Epoch 1/30 - loss: 0.33290
Epoch 2/30 - loss: 0.17791
Epoch 3/30 - loss: 0.15336
Epoch 4/30 - loss: 0.14685
Epoch 5/30 - loss: 0.14342
Epoch 6/30 - loss: 0.14351
Epoch 7/30 - loss: 0.14043
Epoch 8/30 - loss: 0.13967
Epoch 9/30 - loss: 0.13534
Epoch 10/30 - loss: 0.13738
Epoch 11/30 - loss: 0.13614

Epoch 12/30 - loss: 0.13669
Epoch 13/30 - loss: 0.13506
Epoch 14/30 - loss: 0.13443
Epoch 15/30 - loss: 0.13339
Epoch 16/30 - loss: 0.13433
Epoch 17/30 - loss: 0.13514
Epoch 18/30 - loss: 0.13140
Epoch 19/30 - loss: 0.13256
Epoch 20/30 - loss: 0.12823
Epoch 21/30 - loss: 0.12915
Epoch 22/30 - loss: 0.12970
Epoch 23/30 - loss: 0.12694
Epoch 24/30 - loss: 0.12516
Epoch 25/30 - loss: 0.12573
Epoch 26/30 - loss: 0.12480
Epoch 27/30 - loss: 0.12365
Epoch 28/30 - loss: 0.12924
Epoch 29/30 - loss: 0.12379
Epoch 30/30 - loss: 0.12312

جمع‌بندی

در این پروژه، یک مدل سبک برای بهبود تصاویر کم‌نور با الهام از چارچوب GLARE معرفی شد که امکان آموزش و اجرا روی سخت‌افزارهای محدود را فراهم می‌کند. نتایج نشان داد که اگرچه استفاده از تابع هزینه‌ی L1 به بهبود روشنایی تصویر منجر می‌شود، اما به‌تنهایی باعث کاهش اشباع رنگ‌ها می‌گردد. با طراحی یک تابع هزینه‌ی غنی‌تر شامل قیود مربوط به روشنایی، رنگ و اشباع، مدل قادر شد تصاویر کم‌نور را به‌صورت متعادل‌تری بهبود دهد و در عین حفظ جزئیات مکانی، شدت رنگ‌ها را نیز بهتر بازسازی کند. این مطالعه نشان می‌دهد که طراحی آگاهانه‌ی تابع هزینه نقش کلیدی در کیفیت بصری خروجی دارد و می‌تواند جایگزین مناسبی برای افزایش پیچیدگی معماری مدل باشد.